Distance loss in girth-eight cyclic lifted-product codes
An infinite counterfamily with checked Lean proofs

25 October 2026

Abstract

We construct a connected cyclic lifted-product family whose classical Tanner graph has girth exactly eight and whose exponent matrix satisfies neither full row nor full column partition constraint, but whose quantum distance is strictly smaller than its classical distance. For every positive multiple L of eleven, the same fully populated 3×4 exponent matrix gives $25L$ physical qubits and

$$d_Q \leq 11 < 12 \leq d_C \leq 20.$$

The transpose classical code has distance $2L$, so requiring its distance to be at least d_C does not remove the obstruction. The logical operator is pure X, with support in both product blocks; a fiber-sum functional separates it from the entire stabilizer span. A two-parameter template gives such a logical of weight at most eleven for every positive cyclic length, including even lengths and parameters with cancelling terms. Classical lower bounds pass from the eleven-fold base through a parity projection and a stopping-set bound. The results refute a proposed girth/partition sufficiency test motivated by Raveendran, Declercq and Vasić's distance-equality question; their general sufficient-condition question remains open here. Every proof in the manuscript has a checked Lean counterpart, including the source-matrix and full-span translations.

1 The result and the source question

Consider the fixed integer exponent matrix

$$E = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 5 & 3 \\ 0 & 2 & 10 & 7 \end{pmatrix}. \tag{1}$$

At cyclic length L, each entry is reduced modulo L and specifies a circulant permutation, so an exponent zero is an identity block. Let C be its $3L \times 4L$ binary classical check matrix. We use the symmetric single-base lifted product of Raveendran, Declercq and Vasić [1, Eq. (9)], with precise conventions in Section 2.

Theorem 1.1 (Connected counterfamily)**.** *For every positive integer L divisible by eleven, the construction from (1) has the following properties:*

1. *The classical Tanner graph is connected and has girth exactly eight.*
2. *Neither full row nor full column RCPC of [1, Definitions 2–5] holds.*
3. *Its classical and quantum distances satisfy $d_Q \leq 11 < 12 \leq d_C \leq 20$.*
4. *Every nonzero word in $\ker C^{\mathsf{T}}$ has weight $2L$, and such a word exists. In particular, the transpose-code distance is at least d_C.*

The proof is assembled in Sections 3–5. Its decisive features are an explicit logical in the full stabilizer quotient and a uniform classical lower bound. No exact value of d_Q for the whole family is asserted. The bounded distances also mean that this is a finite-length obstruction family, rather than an asymptotically good code family.

Raveendran, Declercq and Vasić's preprint, first posted on 10 March 2025, asks in Section VI for sufficient constraints under which the quantum distance equals the distance of the underlying classical code. Their partition constraints identify a low-weight obstruction at the stabilizer generator scale. The question motivates testing whether classical girth and exclusion of those full partitions already suffice. Theorem 1.1 answers that test negatively. The test is our proposed criterion; it is not a conjecture attributed to the source authors.

Question	Source and precise locator	Answer in this paper
Which constraints guarantee $d_Q = d_C$?	Raveendran–Declercq–Vasić, arXiv:2503.07567v1, 10 March 2025, Section VI; construction Eq. (9).	Still unresolved in general. The counterfamily rules out the particular tests in the next two rows.
Do classical girth at least eight and absence of full RCPC suffice?	Our diagnostic follow-up to that source question; RCPC as in Definitions 2–5 of the same version.	No. Theorem 1.1 gives an infinite family with $d_Q \leq 11 < 12 \leq d_C$.
Does additionally excluding a lighter transpose code suffice?	Our strengthened diagnostic follow-up.	No. The same family has transpose distance $2L \geq d_C$ (Proposition 5.2).

The source's Theorem 2 gives an RCPC implication; its Section V-B and conclusion also describe a recipe excluding stabilizer-weight logicals. Our weight-eleven obstruction does not refute that scale-seven exclusion for a 3×4 base. Instead, it shows why removing that obstruction need not reach the larger classical distance.

Relation to earlier graph questions

Freij-Hollanti, Morris and Šapokaitė [2] study the Tanner graphs of this single-base construction, prove sector graph isomorphism and connectivity results, and ask in Section 5 for sufficient girth-eight conditions and their consequences. Earlier project work answers the girth criterion using rectangle voltages. That local graph information does not determine the minimum of the stabilizer quotient. Here the classical graph passes the short-cycle tests and is connected, while a logical supported in both product blocks still reduces distance. The connection is therefore mathematical, but the invariant and the answered diagnostic are different.

The stopping ingredient is classical. Kelley, Sridhara, Xu and Rosenthal [3, Theorem III.1 and the following note] give the tree bound and its stopping-set interpretation. Their graph-cover description also explains why the occupied image of a lifted codeword is a stopping set. We provide checked versions of the degree-three, girth-eight case and the parity-projection argument needed here; we do not claim these classical principles as new.

Two more recent distance results address different implications. Bhardwaj et al. [4, Appendix G, Theorem 8] give an upper bound with a full binary row-rank premise; this does not supply the lower bound required for $d_Q = d_C$. Hirasaki and Lee [5, Corollary 6] compare quantum distances before and after a compatible group-extension lift. Their odd-index upper bound and their reverse bound with an additional unchanged-logical-dimension premise compare quantum codes with quantum codes, rather

than the quantum and classical distances in the present question. Neither result is used as a premise of our counterfamily proof.

For the central sources, the chronology to the manuscript date is:

Source	Earliest relevant date	Inspected milestone	Elapsed to 25 Oct. 2026
Raveendran et al. [1]	10 March 2025	arXiv v1, same date	594 days
Freij-Hollanti et al. [2]	10 June 2026	arXiv v1, same date	137 days
Kelley et al. [3]	2004 (year precision)	ISIT 2004 proceedings	About 22 years
Bhardwaj et al. [4]	30 July 2026	v1; internal date 24 Aug. 2026	87 days; 62 from internal date
Hirasaki–Lee [5]	30 July 2026	v1; internal date 24 Aug. 2026	87 days; 62 from internal date

These are source ages, not measurements of how long a problem has been unsolved. Appendix G explains the date convention and the bounded literature, GitHub and X audit.

2 The actual source matrices and distances

Write $G_L = \mathbb{Z}/L\mathbb{Z}$ and $R_L = \mathbb{F}_2[x, x^{-1}]/(x^L - 1)$. For $e \in G_L$ put

$$P^e(t,s) = \begin{cases} 1 & t = s + e, \\ 0 & \text{otherwise}, \end{cases} \qquad t, s \in G_L.$$

Binary expansion replaces x^e by P^e. With $B_{ij} = x^{E_{ij}}$, set $B^* = \overline{B}^{\mathsf{T}}$, where $\overline{x^e} = x^{-e}$. The matrices are

$$C = \operatorname{expand}(B), \quad H_{\mathsf{X}} = \operatorname{expand}[B \otimes I_4 \mid I_3 \otimes B^*], \quad H_{\mathsf{Z}} = \operatorname{expand}[I_4 \otimes B \mid B^* \otimes I_3]. \tag{2}$$

The Kronecker products are taken in the ring before expansion. Thus both quantum check matrices have $12L$ rows and $25L$ columns: there is one cyclic fiber.

Number rows and columns from zero. A quantum word has blocks $u(j,k,s)$, $j,k < 4$, and $v(l,i,s)$, $l,i < 3$, represented by matrices $U \in R_L^{4\times 4}$, $V \in R_L^{3\times 3}$. All weights count nonzero binary coefficients across both blocks. The Z check at (j,i,t) evaluates to

$$\sum_{k<4} u(j,k,t - E_{ik}) + \sum_{l<3} v(l,i,t + E_{lj}). \tag{3}$$

Consequently the pure-X kernel equation is

$$UB^{\mathsf{T}} = \overline{B}^{\mathsf{T}} V. \tag{4}$$

For arbitrary binary check coefficients $w(i,j,s)$, the full X stabilizer span consists exactly of the pairs

$$u(i,j,s) = \sum_{k<3} w(k,j,s + E_{ki}), \tag{5}$$

$$v(i,j,s) = \sum_{k<4} w(i,k,s - E_{jk}). \tag{6}$$

Here the index i in (5) ranges below four; in (6) it ranges below three. These formulas quantify over all lifted-check coefficients.

Proposition 2.1 (Matrix and quotient bridges)**.** *The matrices (2) satisfy $H_{\mathsf{Z}} H_{\mathsf{X}}^{\mathsf{T}} = 0$. The H_{Z} check action and full H_{X} row span are exactly (3)–(6). For the fixed single-base family, transposing both square qubit blocks exchanges the two CSS logical sectors and preserves coefficient weight.*

Proof. Expand each monomial using $t = s+e$. Summing first over the single nonzero circulant entry gives (3) and the two span formulas. Substitution of (5)–(6) into (3) cancels the two double sums, proving commutation for arbitrary check coefficients and hence for every pair of check rows. Transposing the two square coordinate blocks and the check type exchanges the X and Z coefficient matrices. This is a coordinate permutation, so it preserves weight and transports the entire spans. The cyclic coordinate is unchanged. The checked matrix identities and equivalences are `SourceModel.inSpanX_iff`, `SourceModel.inSpanZ_iff`, `SourceWordBridge.css_commutation` and `SourceSymmetry.sector_weights`. □

Let d_C be the minimum weight of a nonzero word in $\ker C$. For d_Q use the minimum of the two pure CSS logical spectra,

$$(\ker H_{\mathsf{Z}} \setminus \operatorname{row} H_{\mathsf{X}}) \ \cup \ (\ker H_{\mathsf{X}} \setminus \operatorname{row} H_{\mathsf{Z}}),$$

in the physical binary coordinates. The formal definitions use minima over all words, with explicit existence witnesses; the quantum quotient is nonzero. Our bound also applies directly to usual Pauli distance, because its exhibited operator is a nontrivial pure-X Pauli operator. The translation of the actual source minima is checked in `SourceWordBridge.quantum_distance_eq` and `SourceWordBridge.classical_distance_eq`.

3 A persistent logical supported in both blocks

The obstruction is more transparent in two parameters. For $a, b \in G_L$ define

$$E(a,b) = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & a & b & 3a \\ 0 & 2a & 2b & b+2a \end{pmatrix}. \tag{7}$$

The fixed family is $a = 1, b = 5$. For this template put

$$U = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & x^{-2a} & 0 & 0 \\ 0 & 0 & x^{-2b} & 0 \\ x^{-2a} & x^{-4a} + x^{-2a} & x^{-(b+2a)} & x^{-4a} \end{pmatrix}, \qquad V = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}. \tag{8}$$

Theorem 3.1 (Universal template obstruction)**.** *For every positive L and every $a, b \in G_L$, the pair (8) is in $\ker H_{\mathsf{Z}} \setminus \operatorname{row} H_{\mathsf{X}}$ for the actual matrices of (7), and has weight at most eleven.*

Proof. Direct multiplication gives (4). The first three rows on both sides are, respectively,

$$(1,1,1), \quad (x^{-2a}, x^{-a}, 1), \quad (x^{-2b}, x^{-b}, 1).$$

The last rows agree after cancellation in characteristic two; their three entries are

$$x^{-(b+2a)}, \qquad x^{-3a}, \qquad 1.$$

These identities hold with formal monomials and therefore also after any cyclic quotient. They are checked in `ParametricRing.mixed_cycle` and, directly in coefficients, `ParametricWords.witness_kernel`.

To exclude every stabilizer combination, define

$$\ell(u) = \sum_{s \in G_L} u(0,0,s) + \sum_{s \in G_L} u(1,0,s) \in \mathbb{F}_2. \tag{9}$$

For a span word (5), each of the two sums is $\sum_{k,s} w(k,0,s)$, since translation permutes G_L. They cancel. For (8) the first sum is one and the second zero, so $\ell(U) = 1$. Thus the word lies outside the full X

row span. Finally, U contains eight displayed monomial terms and V three. Their coefficient support is contained in a list of length eleven. Coinciding terms may cancel, and cannot increase the weight. The source-matched conclusion, including matrix commutation, is `ParametricWords.source_parametric_obstruction`. □

The word is pure X throughout. Its use of both product blocks is distinct from combining X and Z Pauli components. This distinction matters: a search confined to one product block would miss the obstruction.

For the fixed exponents, the row-pairing equality $E_{0c} + E_{2c} = 2E_{1c}$ holds in the first three columns but fails in the fourth. The last row of U compensates for that failure. Absence of a *full* partition therefore does not remove all partial-pairing logical mechanisms.

4 The classical lower bound

For an exponent array e over a finite cyclic group A, write a classical word as $z(j, s)$, with boundary

$$(\partial_e z)(i, t) = \sum_{j<4} z(j, t - e_{ij}).$$

A stopping set means a nonempty variable set for which every check has either zero or at least two selected neighbors.

Lemma 4.1 (Cover projection bound)**.** *Let $p : A \to B$ be an additive map of finite cyclic groups and $e' = p \circ e$. Suppose every nonzero word of $\ker \partial_{e'}$ has weight at least d, and every stopping set of its Tanner incidence has size at least s. Then every nonzero word of $\ker \partial_e$ has weight at least $\min(d, 2s)$.*

Proof. For $z \neq 0$ in the upstairs kernel, define its binary parity projection and its occupied image by

$$y(j, r) = \sum_{t:p(t)=r} z(j, t), \qquad T = \{(j, p(t)) : z(j, t) \neq 0\}.$$

Projection commutes with the boundary, so $\partial_{e'} y = 0$. Also $\mathrm{wt}(y) \leq \mathrm{wt}(z)$. If $y \neq 0$, the code bound gives $\mathrm{wt}(z) \geq d$.

If $y = 0$, each occupied fiber contributes a positive even number of ones, hence $\mathrm{wt}(z) \geq 2|T|$. The set T is a stopping set: choose any lifted selected neighbor of a touched projected check. The lifted kernel equation supplies a second selected neighbor at that check. A monomial row has only one neighbor of each column type, so the two neighbors have different column types and remain distinct after projection. Thus $|T| \geq s$, proving the second bound. This argument is `CoverDistance.lower_bound`; `CyclicCover.projected_support_stopping` and `CoverWeight.zero_push_weight_doubles` are its two essential cancellation bridges. □

The use of T as well as y is necessary. A parity projection may vanish even though the occupied image is nonempty, especially in even covers. The proof assumes neither odd cover degree nor unchanged rank.

Lemma 4.2 (The eleven-fold base)**.** *For (1) at $L = 11$, every nonzero classical kernel word has weight at least twelve, and every stopping set has size at least six.*

Proof. For the code bound, the finite certificate reconstructs every 44-bit source-kernel word as an encoding of thirteen free bits. All $2^{13} = 8192$ inputs are checked: the encoded word is zero or its population count is at least twelve. Reconstruction is proved from the actual source rows; it is not an assumption that a displayed list spans the kernel. The bit packing, dot-product and population-count bridges identify this count with coefficient weight. The checked chain is `FamilyBase11.kernel_reco`

nstruct, FamilyClassical11.encoded_lower, FamilyClassical11.actual_classical_lower, and FamilyBaseGraph.code_lower_twelve.

For the stopping bound, the base has variable degree three and no four- or six-cycle, as checked by its finite voltage tests. Choose a variable v in a stopping set. Its three check neighbors have selected partners w_0, w_1, w_2, distinct by exclusion of four-cycles. At w_0, choose two other checks and their selected partners x, y. Four-cycle exclusion prevents $x = y$ or either partner being v; six-cycle exclusion prevents them being w_1, w_2. Together with w_0 being distinct, this yields the six distinct variables v, w_0, w_1, w_2, x, y. This is the degree-three, girth-eight Tanner-tree bound credited above. Its general checked version is TannerSix.stopping_lower_six; its actual base incidence application is FamilyTreeStopping.base_stopping_six. □

Corollary 4.3 (Uniform classical lower bound)**.** *If $L > 0$ and $11 \mid L$, the classical code of (1) has $d_C \geq 12$.*

Proof. Use reduction $G_L \to G_{11}$ in Lemma 4.1, with $d = 12$ and $s = 6$ from Lemma 4.2. The exponent array reduces to the eleven-fold base and $\min(12, 2 \cdot 6) = 12$. The source-action bridge is SourceDistance.c_action; the application is FamilyCoverAlternative.classical_lower_twelve. □

Corollary 4.4 (Arithmetic extensions)**.** *Let $L > 0$, and suppose an additive map $p : G_L \to G_{11}$ sends a to 1 and b to 5. Then the template (7) has no classical four- or six-cycle, has neither full RCPC, and satisfies $d_Q \leq 11 < 12 \leq d_C$. In particular this holds when $11 \mid L$ and a, b reduce to $1, 5$ modulo eleven.*

Proof. The exponent array projects to (1) modulo eleven. The same cover bound proves the classical inequality, and Theorem 3.1 supplies the logical. A four-cycle, six-cycle or full partition upstairs would project to a forbidden voltage or partition equality at eleven; the base exclusions are specified in Section 5. The complete arithmetic implication is ParametricProjection.projected_counterfamily, with the stated congruence specialization in ParametricProjection.cyclic_congruence_counterfamily. □

We assert connectedness, the bound $d_C \leq 20$ and transpose distance $2L$ only for the fixed specialization of Theorem 1.1.

5 Upper bound, transpose code and graph geometry

Proposition 5.1 (A classical word of weight at most twenty)**.** *For the fixed family with $L > 0$ and $11 \mid L$, $d_C \leq 20$.*

Proof. The following column vector is a polynomial kernel word for B:

$$\begin{pmatrix} x^4 + x^5 + x^7 + x^8 + x^9 + x^{11} \\ x^2 + x^3 + x^6 + x^7 + x^8 + x^{11} \\ 1 + x^2 + x^3 + x^6 \\ 1 + x^4 + x^5 + x^9 \end{pmatrix}. \tag{10}$$

Multiplying by each row of B cancels all terms in characteristic two. There are twenty displayed monomials, so the coefficient weight is at most twenty after reduction. The third component is nonzero: its exponents $0, 2, 3, 6$ remain distinct after reduction modulo eleven, and hence cannot cancel upstairs. The coefficient kernel identity, nonzero bridge and support bound are proved in ClassicalUpper20.classical_distance_upper. □

Proposition 5.2 (Exact transpose distance)**.** *For the fixed matrix (1) and every positive L, the words of $\ker C^{\mathsf{T}}$ are exactly the triples of constant fiber functions (z_0, z_1, z_2) with $z_0 + z_1 + z_2 = 0$. Every nonzero such word has weight $2L$, and a nonzero such word exists.*

Proof. The actual transpose equations, with their plus sign, are

$$f_j(t) := \sum_{i<3} z_i(t + E_{ij}) = 0 \quad (j < 4).$$

There is a direct characteristic-two identity

$$\sum_{j=0}^{3} \sum_{r \in A_j} f_j(t + r) = z_2(t + 1) + z_2(t + 2), \tag{11}$$

where

$$A_0 = \{1, 4, 6, 7, 8, 9\}, \quad A_1 = \{0, 2, 4, 8, 9\},$$
$$A_2 = \{3, 4\}, \qquad A_3 = \{0, 1, 2, 3, 4, 6, 7\}.$$

Thus z_2 has period one and is constant. The equations $f_0 = f_1 = 0$ then make z_1 constant; $f_0 = 0$ makes z_0 constant as well and gives their sum zero. Conversely such a constant triple satisfies every equation. A nonzero even binary triple has exactly two ones, giving weight $2L$; $(1, 1, 0)$ supplies existence. Identity (11) is `FamilyTranspose.third_period`; the characterization and count are `FamilyTranspose.kernel_iff` and `FamilyTranspose.distance_characterization`. The actual transpose action is checked in `DistanceFamily.transpose_c_action`. □

For the comparison $d_C \leq 2L$ at every positive L, set two entire classical variable fibers to one and the other two to zero. Each classical check then sees two ones. This word has weight $2L$; `DistanceFamily.classical_distance_upper` checks the kernel, nonzero and weight claims. Consequently no nonzero transpose word is lighter than d_C, as assembled in `DistanceFamily.stronger_counterfamily`.

Proposition 5.3 (Classical graph and full partitions)**.** *For $L > 0$ divisible by eleven, the classical graph of (1) is connected, has girth exactly eight and has neither full RCPC.*

Proof. A classical incidence joins variable (j, s) to check $(i, s + E_{ij})$. A simple four-cycle has two distinct row types and two distinct column types and forces

$$E_{ic} - E_{jc} + E_{jd} - E_{id} = 0.$$

A simple six-cycle has three distinct row and column types and forces

$$E_{ic} - E_{jc} + E_{jd} - E_{kd} + E_{kf} - E_{if} = 0.$$

Distinctness of the types follows from the unique neighbor of any specified type at a monomial incidence; the graph/cycle bridge is `VoltageCycles.no_four` and `VoltageCycles.no_six`. The finite tests at eleven show that neither voltage can vanish with the indicated distinct types. Any equality upstairs reduces to one at eleven, so both cycle lengths are excluded for every $11 \mid L$.

There is an explicit eight-cycle, written alternately with check vertices c and variable vertices v:

$$c_{0,0},\ v_{0,0},\ c_{1,0},\ v_{1,-1},\ c_{2,1},\ v_{0,1},\ c_{1,1},\ v_{1,0},\ c_{0,0}.$$

Each displayed edge satisfies the incidence equation. The four variables and four checks are separately distinct since $L \geq 11$. The graph is simple and bipartite, so its girth is exactly eight. The voltage exclusions and distinctness are `FamilyGirth.exact_eight_certificate`; all eight edges are checked in `FamilyGirth.eight_edges`.

For three rows a full row RCPC means that distinct i, j, k satisfy $E_{ic} + E_{jc} = 2E_{kc}$ in every column, including the odd-size self-pair convention. For four columns a full column RCPC partitions the columns

into two pairs with equal sums in every row. The finite tests exclude both at eleven; again projection excludes them upstairs. These are `FamilyNoShort.no_row_pairing` and `FamilyNoShort.no_column_pairing`.

Finally the zero row connects all variable types at a common fiber, and the zero column connects the check types. The entry $E_{11} = 1$ gives a walk from variable $(0, s)$ to $(0, s+1)$ through variable $(1, s)$ and check $(1, s+1)$. Iterating reaches every cyclic fiber, so all vertices are connected. This argument works for every positive L and is `FamilyConnected.connected`. □

Proof of Theorem 1.1. Proposition 5.3 proves the graph and RCPC assertions. Theorem 3.1, specialized to $a = 1, b = 5$, gives $d_Q \leq 11$. Corollary 4.3 and Proposition 5.1 give $12 \leq d_C \leq 20$. Proposition 5.2 and the two-fiber classical word give the transpose comparison. The specialization of all source matrices and full logical predicates is checked in `ParametricSpecialization.logicalX`. The source-distance conclusion is `SourceWordBridge.source_distance_drop`. □

6 Formal coverage and reproducibility

The proofs use Lean 4.36.0-rc1 and mathlib commit `5f7b4198d22a9c82e1c698a379334fe128095c82`. The admitted counterfamily closure contains fifty authored modules. All were built from their recorded sources and rebuilt in an isolated directory with original research compiled objects excluded. Pinned cached mathlib, dependency and standard-library objects were reused; this was not a fresh mathlib rebuild. The axiom audit admits only `propext`, `Classical.choice` and `Quot.sound`. Native-decision prototypes and external search exclusions are outside this closure.

Table 2 covers each manuscript proof, including the translations and applications. Appendices A–E explain the formal objects and give complete central Lean proofs, from the stabilizer separator and cover bound to the source-distance conclusion. Appendix F gives current and immutable URLs for all fifty admitted modules. Every theorem link in the text targets its declaration at commit `98eb91cd9049`, whose source bytes match the checked builds. The companion proof guide and source archive provide hashes, logs and editable files in addition to the PDF exposition. Generated appendix listings are exact source extracts with typographical substitutions for Unicode symbols, not new proof modules.

Table 2: Complete proof-to-Lean correspondence. Full names, source hashes, imports and checked logs are linked in the companion guide.

Manuscript proof	Faithful formal result	Objects and scope
Proposition 2.1; source minima	`SourceModel.inSpanX_iff`; `SourceModel.inSpanZ_iff`; `SourceWordBridge.quantum_weight_iff`; `SourceWordBridge.css_commutation`; `SourceSymmetry.sector_weights`	Actual coefficient matrices, arbitrary binary row coefficients, both sectors, all physical words and weights.
Theorem 3.1	`ParametricRing.mixed_cycle`; `ParametricWords.source_parametric_obstruction`	Every positive L, arbitrary $a, b \in G_L$; full row span, cancellation and physical support bound.
Lemma 4.1	`CoverDistance.lower_bound`; `CyclicCover.kernel_push`; `CyclicCover.projected_support_stopping`; `CoverWeight.zero_push_weight_doubles`	Additive projection; nonzero upstairs word; parity and occupied-support cases treated separately.

Manuscript proof	Faithful formal result	Objects and scope
Lemma 4.2, code certificate	`FamilyBase11.kernel_reconstruct`; `FamilyClassical11.actual_classical_lower`; `FamilyBaseGraph.code_lower_twelve`	Every source-kernel word; all thirteen-bit inputs; incidence, dot-product and Hamming-weight bridges.
Lemma 4.2, stopping bound	`TannerSix.stopping_lower_six`; `FamilyTreeStopping.base_stopping_six`	Nonempty stopping sets; degree at least three and no four/six cycles; actual base incidence.
Corollary 4.3	`FamilyCoverAlternative.classical_lower_twelve`; `SourceDistance.c_action`	All $L > 0$, $11 \mid L$; actual classical source words.
Corollary 4.4	`ParametricProjection.projected_counterfamily`; `ParametricProjection.cyclic_congruence_counterfamily`	Explicit additive map sending a, b to $1, 5$; no implicit unit, connectedness or rank premise.
Proposition 5.1	`ClassicalUpper20.kernel`; `ClassicalUpper20.nonzero`; `ClassicalUpper20.classical_distance_upper`	Polynomial word, nonzero under reduction modulo eleven, at most twenty physical coordinates.
Proposition 5.2; two-fiber comparison	`FamilyTranspose.third_period`; `FamilyTranspose.kernel_iff`; `FamilyTranspose.distance_characterization`; `DistanceFamily.stronger_counterfamily`	Actual C^{T}, every positive L, all kernel words and nonempty minimum; comparison with actual d_C.
Proposition 5.3	`VoltageCycles.no_four`; `VoltageCycles.no_six`; `FamilyNoShort.no_row_pairing`; `FamilyNoShort.no_column_pairing`; `FamilyGirth.exact_eight_certificate`; `FamilyGirth.eight_edges`; `FamilyConnected.connected`	Simple-cycle bridges, checked base exclusions, explicit eight-cycle vertices and edges, actual Tanner reachability.
Theorem 1.1, assembly	`DistanceFamily.infinite_counterfamily`; `SourceWordBridge.source_distance_drop`; `ParametricSpecialization.logicalX`; preceding bound/graph theorems	Same source construction and minima, $L > 0$, $11 \mid L$; no exact quantum distance added.

The formal appendix follows the mathematical argument in dependency order: actual source objects and full spans; the logical obstruction; parity projection and the base certificate; graph and transpose properties; and the final all-word distance statement. The dependency diagram in Appendix A separates these branches and their trust boundaries. External distance searches are not premises of the proofs.

7 Implications and the next bounded question

The counterfamily separates local graph and full-partition exclusions from quantum distance preservation. Connectedness rules out explaining it as disjoint copies of the eleven-fold example, and the exact transpose characterization rules out a lighter transpose code as the cause. What remains is a logical

using both product blocks, produced by a partial pairing that full RCPC tests do not detect.

The source's general equality problem is not answered by this negative test. A bounded follow-up is to prove or refute

$$d_Q \geq \min(d_C, 12) \tag{12}$$

for normalized, fully populated 3×4 cyclic exponent matrices whose classical Tanner graph has girth at least eight, which have no full RCPC, whose transpose distance is at least d_C, and which also obey the following partial-pairing exclusion: for every choice of distinct rows i, j, k, no value among

$$E_{ic} + E_{jc} - 2E_{kc}, \qquad c = 0, 1, 2, 3,$$

occurs three or more times. Normalization means the first row and first column are zero; the entry tests are stated in this convention. The fixed counterfamily violates this additional test.

This is a project-derived follow-up, proposed after the counterexample and the bounded diagnostic recorded in the supporting evidence. It is not a proved criterion or a new author-posed conjecture. The remaining obstruction is control of all small logical supports in the full quotient, especially weights eight through eleven. Two separately checked generic lemmas provide structural progress, but neither completes the source-support classification or proves (12). A complete exact-distance certificate for a selected finite instance is a separate alternative. Verifying the source's scale-seven recipe is a source-claim audit, rather than a newly open mathematical problem.

Question answered. The proposed girth/RCPC criterion, even with connectedness and the transpose-distance comparison, is false by Theorem 1.1.

Question still open. General sufficient constraints guaranteeing $d_Q = d_C$ in the source construction.

Next proposed target (follow-up). Prove or refute the truncated bound (12) under the explicit strengthened hypotheses above.

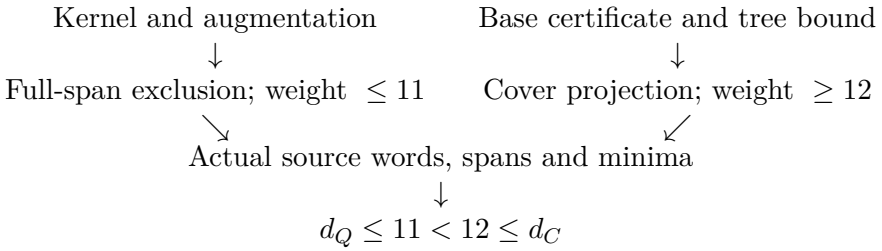
A The checked Lean argument: objects and dependency map

This appendix presents the formal argument inside the paper. It explains how the source objects are represented and prints the complete central declaration bodies used in the distance obstruction, together with the applications needed to recover the mathematical theorems. Appendix F then gives current and immutable links for every admitted source file. These links supplement the displayed proofs; they are also the route to routine coordinate lemmas and the large finite row certificates not printed in full.

All listings below are generated from the checked sources at commit `98eb91cd9049`. They retain complete declarations, with Unicode symbols typeset mathematically and long lines wrapped. They are extracts from their original namespaces and imported contexts, rather than separate standalone Lean files. Unless stated otherwise, cyclic modules use an implicit natural number `L` and the instance `[NeZero L]`. This excludes length zero, not even or composite lengths. The generic cover modules instead use finite abelian groups and the instances stated with their theorem signatures. The archive's `appendix-listings.json` binds each extract to its source hash and exact line ranges. No mathematical proof source was changed to produce these listings.

A.1 How the branches fit together

The lower and upper bounds are proved independently and combined only after their actual source-coordinate translations. The following diagram summarizes the dependency structure; the graph and transpose branches establish the additional clauses of Theorem 1.1.



Branch		Checked dependency chain
Quantum upper bound	$\longrightarrow$	Coefficient kernel → augmentation separator → full-span nonmembership → physical support at most eleven.
Classical lower bound	$\longrightarrow$	Base row reconstruction and finite weight certificate; Tanner-tree stopping bound → parity projection → every nonzero upstairs word has weight at least twelve.
Source distances	$\longrightarrow$	Actual matrix actions and full spans → both CSS sectors and all-word weight predicates → equality of the formal minima → $d_Q < d_C$.
Graph and transpose properties	$\longrightarrow$	Voltage exclusions plus explicit cycle edges; cyclic reachability; transpose period identity → constant fibers → exact transpose weight.

The trust boundary is the ordinary Lean kernel and the recorded standard axioms `propext`, `Classical.choice` and `Quot.sound`. The finite classical certificate is checked by Lean; external C++ and SAT exclusions are absent from these branches. The separately audited generic follow-up lemmas are not used to claim the unproved truncated-distance condition.

A.2 Actual matrices and physical coordinates

`GeneralLP` gives the coefficient matrices for an arbitrary monomial 3×4 exponent array. Its `Qubit` type is the disjoint union of the 4×4 and 3×3 product blocks, each with one cyclic coordinate. The

P coefficient uses exactly $t = s + e$. Thus the definitions implement (2), including the star's negative exponent, without a second lift. The syndrome and two stabilizer actions displayed here expose every sign.

Checked source: GeneralLP.HX; GeneralLP.HZ; GeneralLP.Syndrome; GeneralLP.stabilizerLeft; GeneralLP.stabilizerRight; GeneralLP.InSpanX; GeneralLP.LogicalX.

```
def HX (e : Fin 3 → Fin 4 → ZMod L) (c : XCheck L) (q : Qubit L) : ZMod 2 := match q with
  | .inl ((i,j),s) => if c.1.2=j then P (e c.1.1 i) c.2 s else 0
  | .inr ((i,j),s) => if c.1.1=i then P (-e j c.1.2) c.2 s else 0

def HZ (e : Fin 3 → Fin 4 → ZMod L) (c : ZCheck L) (q : Qubit L) : ZMod 2 := match q with
  | .inl ((i,j),s) => if c.1.1=i then P (e c.1.2 j) c.2 s else 0
  | .inr ((i,j),s) => if c.1.2=j then P (-e i c.1.1) c.2 s else 0

def Syndrome (e : Fin 3 → Fin 4 → ZMod L)
    (u : Fin 4 → Fin 4 → ZMod L → ZMod 2) (v : Fin 3 → Fin 3 → ZMod L → ZMod 2)
    (j : Fin 4) (i : Fin 3) (t : ZMod L) : ZMod 2 :=
  (∑ k : Fin 4, u j k (t-e i k)) + (∑ l : Fin 3, v l i (t+e l j))

def stabilizerLeft (e : Fin 3 → Fin 4 → ZMod L)
    (w : Fin 3 → Fin 4 → ZMod L → ZMod 2) (i j : Fin 4) (s : ZMod L) : ZMod 2 :=
  ∑ k : Fin 3, w k j (s+e k i)

def stabilizerRight (e : Fin 3 → Fin 4 → ZMod L)
    (w : Fin 3 → Fin 4 → ZMod L → ZMod 2) (i j : Fin 3) (s : ZMod L) : ZMod 2 :=
  ∑ k : Fin 4, w i k (s-e j k)

def InSpanX (e : Fin 3 → Fin 4 → ZMod L)
    (u : Fin 4 → Fin 4 → ZMod L → ZMod 2) (v : Fin 3 → Fin 3 → ZMod L → ZMod 2) : Prop
    :=
  ∃ w : XCheck L → ZMod 2, ∀ q, join u v q=∑ c, w c*HX e c q

def LogicalX (e : Fin 3 → Fin 4 → ZMod L)
    (u : Fin 4 → Fin 4 → ZMod L → ZMod 2) (v : Fin 3 → Fin 3 → ZMod L → ZMod 2) : Prop
    :=
  (∀ c : ZCheck L, ∑ q, HZ e c q*join u v q=0) ∧ ¬ InSpanX e u v
```

A.3 The complete row span, not just the generators

The following equivalence quantifies over an arbitrary binary coefficient for every lifted check. Its forward direction evaluates a general row combination in each physical block; its reverse direction reconstructs that combination. The `inSpanZ_iff` theorem in the same file proves the corresponding other-sector statement by transposing both square blocks. This is the bridge needed before an augmentation obstruction can certify a logical class.

Checked source: SourceModel.InSpanX; SourceModel.inSpanX_iff.

```
def InSpanX (u : Fin 4 → Fin 4 → ZMod L → ZMod 2)
    (v : Fin 3 → Fin 3 → ZMod L → ZMod 2) : Prop :=
  ∃ w : XCheck L → ZMod 2, ∀ q, join u v q = ∑ c, w c * HX c q

theorem inSpanX_iff (u : Fin 4 → Fin 4 → ZMod L → ZMod 2)
    (v : Fin 3 → Fin 3 → ZMod L → ZMod 2) :
    InSpanX u v ↔ ∃ w, u = stabilizerLeft w ∧ v = stabilizerRight w := by
  constructor
```

```
  · rintro ⟨w,hw⟩
    refine ⟨(fun i j s => w ((i,j),s)),?_,?_⟩
    · funext i j s
      exact (hw (.inl ((i,j),s))).trans (x_span_left (fun a b t => w ((a,b),t)) i j s)
    · funext i j s
      exact (hw (.inr ((i,j),s))).trans (x_span_right (fun a b t => w ((a,b),t)) i j s)
  · rintro ⟨w,rfl,rfl⟩
    refine ⟨(fun c => w c.1.1 c.1.2 c.2),?_⟩
    intro q
    cases q with
    | inl p => rcases p with ⟨⟨i,j⟩,s⟩; exact (x_span_left w i j s).symm
    | inr p => rcases p with ⟨⟨i,j⟩,s⟩; exact (x_span_right w i j s).symm
```

A.4 The all-word distance predicate

Here `Kernel` and `InSpan` quantify over actual source binary coordinates, while `QuantumHasWeight` includes both logical sectors. These are the predicates behind the minima, rather than a restricted search through displayed words. Existence and the minimum equivalences are used in Appendix E.

Checked source: SourceWordBridge.Kernel; SourceWordBridge.InSpan; SourceWordBridge.QuantumHasWeight.

```
def Kernel {Row : Type*} [Fintype Row] (H : Row → Qubit L → ZMod 2) (w : Qubit L → ZMod 2)
    :=
  ∀ c, ∑ q, H c q * w q = 0

def InSpan {Row : Type*} [Fintype Row] (H : Row → Qubit L → ZMod 2) (w : Qubit L → ZMod 2)
    :=
  ∃ a : Row → ZMod 2, ∀ q, w q = ∑ c, a c * H c q

def QuantumHasWeight (L : Nat) [NeZero L] (n : Nat) : Prop :=
  ∃ w : Qubit L → ZMod 2,
    ((Kernel HZ w ∧ ¬ InSpan HX w) ∨ (Kernel HX w ∧ ¬ InSpan HZ w)) ∧
      (CoverWeight.support w).card = n
```

B The logical obstruction: kernel, full-span separation and weight

This branch proves Theorem 3.1. The coefficient word is defined for arbitrary parameters in `ZMod L`. In particular, monomial terms may coincide, and the weight argument must allow characteristic-two cancellation.

B.1 The word and its full coefficient kernel proof

The matrix literal below is the coefficient version of (8). `delta a s` is one exactly when $s = a$. The proof checks all twelve check types with arbitrary fiber coordinate; it does not sample lift sizes.

Checked source: ParametricWords.e; ParametricWords.U; ParametricWords.V; ParametricWords.witness_kernel.

```
def e (a b : ZMod L) : Fin 3 → Fin 4 → ZMod L :=
  !![0,0,0,0;0,a,b,3*a;0,2*a,2*b,b+2*a]

def U (a b : ZMod L) (i j : Fin 4) (s : ZMod L) : ZMod 2 :=
  !![delta 0 s,0,0,0;
     0,delta (-2*a) s,0,0;
     0,0,delta (-2*b) s,0;
     delta (-2*a) s,delta (-4*a) s+delta (-2*a) s,delta (-(b+2*a)) s,delta (-4*a) s] i j

abbrev V : Fin 3 → Fin 3 → ZMod L → ZMod 2 := FamilyWords.V

theorem witness_kernel (a b : ZMod L) (j : Fin 4) (i : Fin 3) (t : ZMod L) :
    GeneralLP.Syndrome (e a b) (U a b) V j i t=0 := by
  have ht : (2 : ZMod 2)=0 := by decide +kernel
  have hf : (4 : ZMod 2)=0 := by decide +kernel
  fin_cases j <;> fin_cases i <;>
    norm_num [GeneralLP.Syndrome,e,U,V,FamilyWords.V,Fin.sum_univ_succ,delta_sub,delta_add]
  all_goals ring_nf
  all_goals simp only [ht,hf,mul_zero,sub_zero,zero_sub,neg_zero,add_zero,zero_add]
```

B.2 Why the separator annihilates every stabilizer

The augmentation is the two fiber sums of (9). Translation permutes a finite cyclic fiber, so both terms of a stabilizer give the same sum and cancel. The second theorem extracts a left-block stabilizer representation from membership in the entire physical row span. Together these statements make the separator apply to arbitrary source stabilizer combinations.

Checked source: GeneralLP.augmentation_stabilizer; GeneralLP.full_span_left.

```
theorem augmentation_stabilizer (e : Fin 3 → Fin 4 → ZMod L)
    (w : Fin 3 → Fin 4 → ZMod L → ZMod 2) :
    FamilyWords.augmentation (stabilizerLeft e w)=0 := by
  have h (i : Fin 4) : (∑ s, stabilizerLeft e w i 0 s)=∑ k, ∑ s, w k 0 s := by
    unfold stabilizerLeft
    rw [Finset.sum_comm]
    simp_rw [FamilyWords.sum_shift]
  unfold FamilyWords.augmentation
  rw [h 0,h 1]
  exact CharTwo.add_self_eq_zero _

theorem full_span_left (e : Fin 3 → Fin 4 → ZMod L)
```

```
    (u : Fin 4 → Fin 4 → ZMod L → ZMod 2) (v : Fin 3 → Fin 3 → ZMod L → ZMod 2)
    (h : InSpanX e u v) : ∃ w, u=stabilizerLeft e w := by
  obtain ⟨w,hw⟩ := h
  refine ⟨(fun i j s => w ((i,j),s)),?_⟩
  funext i j s
  exact (hw (.inl ((i,j),s))).trans (x_span_left e (fun a b t => w ((a,b),t)) i j s)
```

B.3 Nontriviality of the actual source logical

The source check action transports the coefficient identity to the matrix kernel. If the word belonged to the stabilizer span, its left block would have augmentation zero, contradicting the computed value one. The following is the complete proof of that nonmembership, with no finite span search.

Checked source: ParametricWords.augmentation_witness; ParametricWords.actual_logical.

```
theorem augmentation_witness (a b : ZMod L) : FamilyWords.augmentation (U a b)=1 := by
  simp [FamilyWords.augmentation,U,delta]

theorem actual_logical (a b : ZMod L) : GeneralLP.LogicalX (e a b) (U a b) V := by
  constructor
  · intro c
    exact (GeneralLP.z_action (e a b) (U a b) V c.1.1 c.1.2 c.2).trans (witness_kernel a b _
    _ _)
  · intro h
    obtain ⟨w,hw⟩ := GeneralLP.full_span_left (e a b) (U a b) V h
    have h1 := augmentation_witness a b
    rw [hw,GeneralLP.augmentation_stabilizer] at h1
    exact zero_ne_one h1
```

B.4 Physical Hamming weight, including cancelling parameters

The support list consists of eleven physical qubit coordinates. The first proof shows the word vanishes outside that list; the second compares finite support cardinalities. It deliberately uses list length as an upper bound, so duplicate coordinates and cancellation cannot invalidate the theorem.

Checked source: ParametricWords.supportList; ParametricWords.zero_outside; ParametricWords.weight_le.

```
def supportList (a b : ZMod L) : List (Qubit L) :=
  [.inl ((0,0),0),.inl ((1,1),-2*a),.inl ((2,2),-2*b),
   .inl ((3,0),-2*a),.inl ((3,1),-4*a),.inl ((3,1),-2*a),
   .inl ((3,2),-(b+2*a)),.inl ((3,3),-4*a),
   .inr ((0,2),0),.inr ((1,1),0),.inr ((2,0),0)]

theorem zero_outside (a b : ZMod L) (q : Qubit L) (hq : q∉supportList a b) :
    GeneralLP.join (U a b) V q=0 := by
  cases q with
  | inl p =>
    rcases p with ⟨⟨i,j⟩,s⟩
    fin_cases i <;> fin_cases j <;>
      simp_all [GeneralLP.join,SourceModel.join,U,supportList,delta]
  | inr p =>
    rcases p with ⟨⟨i,j⟩,s⟩
    fin_cases i <;> fin_cases j <;>
      simp_all [GeneralLP.join,SourceModel.join,V,FamilyWords.V,supportList,delta]
```

```
theorem weight_le (a b : ZMod L) :
    (CoverWeight.support (GeneralLP.join (U a b) V)).card≤11 := by
  have hs : CoverWeight.support (GeneralLP.join (U a b) V) ⊆ (supportList a b).toFinset := by
    intro q hq
    have hn : GeneralLP.join (U a b) V q≠0 := (Finset.mem_filter.mp hq).2
    by_contra h
    have hh : q∉supportList a b := by simpa using h
    exact hn (zero_outside a b q hh)
  have h := Finset.card_le_card hs
  have hl := List.toFinset_card_le (l := supportList a b)
  have he : (supportList a b).length=11 := rfl
  omega
```

B.5 The source-matched universal conclusion

This theorem assembles actual CSS matrix commutation, full logical nontriviality and physical coefficient weight. It is the same universal parameter statement used in Theorem 3.1.

Checked source: ParametricWords.source_parametric_obstruction.

```
theorem source_parametric_obstruction (a b : ZMod L) :
    (∀ c : GeneralLP.XCheck L, ∀ d : GeneralLP.ZCheck L,
      (∑ q, GeneralLP.HZ (e a b) d q * GeneralLP.HX (e a b) c q)=0) ∧
    GeneralLP.LogicalX (e a b) (U a b) V ∧
    (CoverWeight.support (GeneralLP.join (U a b) V)).card≤11 :=
  ⟨GeneralLP.css_commutation (e a b),parametric_obstruction a b⟩
```

C The classical lower bound: projection and the finite base

This branch separates ordinary binary parity projection from the image of occupied support. The distinction is necessary when all projected parities cancel, and avoids an odd-cover assumption.

C.1 Cancellation gives two occupied coordinates per fiber

The push-forward is a sum in `ZMod 2`. If it is zero, a nonempty fiber support cannot have cardinality one. Summing the resulting two-per-fiber bound gives twice the size of the occupied image. The full proof below works for arbitrary finite source and target types, not just cyclic groups.

Checked source: CoverWeight.push; CoverWeight.zero_push_fiber_lower; CoverWeight.zero_push_weight_doubles.

```
def push (p : A → B) (x : A → ZMod 2) (b : B) : ZMod 2 :=
  ∑ a, if p a = b then x a else 0

theorem zero_push_fiber_lower (p : A → B) (x : A → ZMod 2) (hz : push p x = 0)
    (b : B) (hb : b ∈ (support x).image p) :
    2 ≤ ((support x).filter (fun a => p a = b)).card := by
  obtain ⟨a,ha,hp⟩ := Finset.mem_image.mp hb
  have hpos : 0 < ((support x).filter (fun a => p a = b)).card :=
    Finset.card_pos.mpr ⟨a,Finset.mem_filter.mpr ⟨ha,hp⟩⟩
  have hc := push_eq_card p x b
  have he := congrFun hz b
  rw [he] at hc
  have hne : ((support x).filter (fun a => p a = b)).card ≠ 1 := by
    intro h1
    rw [h1] at hc
    norm_num at hc
  omega

theorem zero_push_weight_doubles (p : A → B) (x : A → ZMod 2) (hz : push p x = 0) :
    2 * ((support x).image p).card ≤ (support x).card := by
  calc
    2 * ((support x).image p).card = ∑ b ∈ (support x).image p, 2 := by simp [Nat.mul_comm]
    _ ≤ ∑ b ∈ (support x).image p, ((support x).filter (fun a => p a = b)).card :=
      Finset.sum_le_sum (fun b hb => zero_push_fiber_lower p x hz b hb)
    _ = (support x).card := (Finset.card_eq_sum_card_image p (support x)).symm
```

C.2 The occupied image is a nonempty stopping set

The projected support theorem uses the actual monomial incidence. At a touched lifted check, binary parity gives a second selected neighbor. Its base column type differs from the first, so the neighbors remain distinct after projection. The helper `stopping_of_partners` turns these partners into the finite-set stopping predicate.

Checked source: CyclicCover.projected_support_stopping.

```
theorem projected_support_stopping (p : A →+ B) (e : Fin 3 → Fin 4 → A)
    (x : Word A) (hx : Kernel e x) (hn : x ≠ 0) :
    SmallStopping.Stopping (Adjacent (fun i j => p (e i j)))
      ((CoverWeight.support x).image (project p)) := by
  have hs : (CoverWeight.support x).Nonempty := by
    by_contra h
```

```
    apply hn
    funext v
    by_contra hv
    change x v ≠ 0 at hv
    exact h ⟨v,Finset.mem_filter.mpr ⟨Finset.mem_univ _,hv⟩⟩
  apply stopping_of_partners _ _ (hs.image _)
  intro v hv c hc
  obtain ⟨w,hw,rfl⟩ := Finset.mem_image.mp hv
  have hwnz : x w ≠ 0 := (Finset.mem_filter.mp hw).2
  let d : Fin 3 × A := (c.1,w.2+e c.1 w.1)
  have hk : ∃ k : Fin 4, k ≠ w.1 ∧ x (k,d.2-e c.1 k) ≠ 0 := by
    by_contra h
    push Not at h
    have he : boundary e x d = x w := by
      unfold boundary
      rw [Finset.sum_eq_single w.1]
      · simp [d]
      · intro k hk hne
        exact h k hne
      · simp
    exact hwnz (he.symm.trans (hx d))
  obtain ⟨k,hkw,hkx⟩ := hk
  refine ⟨project p (k,d.2-e c.1 k),Finset.mem_image.mpr ⟨_,by simp [CoverWeight.support,hkx],
    rfl⟩,?_,?_⟩
  · intro h
    exact hkw (congrArg Prod.fst h)
  · simpa [Adjacent,project,d,map_sub,map_add] using hc
```

C.3 The complete cover-bound theorem

Both alternatives now appear in a single full theorem and proof. When the parity projection is nonzero, its base code distance supplies d; when it is zero, the occupied image supplies $2s$. The printed hypotheses make clear which base facts are required.

Checked source: CoverDistance.lower_bound.

```
theorem lower_bound (p : A →+ B) (e : Fin 3 → Fin 4 → A) (d s : Nat)
    (hcode : ∀ y : CyclicCover.Word B, CyclicCover.Kernel (fun i j => p (e i j)) y →
      y≠0 → d≤(CoverWeight.support y).card)
    (hstop : ∀ T : Finset (Fin 4 × B), SmallStopping.Stopping
      (CyclicCover.Adjacent (fun i j => p (e i j))) T → s≤T.card)
    (x : CyclicCover.Word A) (hx : CyclicCover.Kernel e x) (hn : x≠0) :
    min d (2*s) ≤ (CoverWeight.support x).card := by
  let y := CoverWeight.push (CyclicCover.project p) x
  by_cases hy : y=0
  · have hs := hstop _ (CyclicCover.projected_support_stopping p e x hx hn)
    have hw := CoverWeight.zero_push_weight_doubles (CyclicCover.project p) x hy
    exact (Nat.min_le_right d (2*s)).trans ((Nat.mul_le_mul_left 2 hs).trans hw)
  · have hd := hcode y (CyclicCover.kernel_push p e x hx) hy
    have hw := CoverWeight.push_weight_le (CyclicCover.project p) x
    change (CoverWeight.support y).card ≤ (CoverWeight.support x).card at hw
    exact (Nat.min_le_left d (2*s)).trans (hd.trans hw)
```

C.4 What the eleven-fold certificate actually checks

`FamilyEnum11.encode` is an explicit XOR of thirteen 44-bit vectors. `FamilySource11` proves `kernel_reconstruct`: every actual source-kernel vector equals that encoding at its thirteen extracted free coordinates. It is an identity derived from the source rows, not an assumed generator-span description. `FamilyEnum11.goodBlock` checks 128 consecutive inputs, and `FamilyDistance11.all_blocks` checks all 64 blocks. Their complete source files, including the bit constants and row identities, are linked in Appendix F.

The following proofs turn the checked blocks into a theorem for an arbitrary thirteen-bit input and then for every source-kernel vector. This displays the certificate's soundness step rather than an external enumerator result.

Checked source: `FamilyClassical11.encoded_lower`; `FamilyClassical11.actual_classical_lower`.

```
theorem encoded_lower (c : BitVec 13) : encode c = 0 ∨ 12 ≤ (encode c).cpop.toNat := by
  have hc : c.toNat < 8192 := c.isLt
  have hb : c.toNat / 128 < 64 := by omega
  have h := all_blocks ⟨c.toNat / 128, hb⟩
  unfold goodBlock at h
  have hh := List.all_eq_true.mp h (c.toNat % 128) (List.mem_range.mpr (Nat.mod_lt _ (by
    decide)))
  have he : 128*(c.toNat/128)+c.toNat%128 = c.toNat := by omega
  rw [he, BitVec.ofNat_toNat] at hh
  have hp : encode c = 0#44 ∨ (12 : BitVec 44) ≤ (encode c).cpop := by simpa using hh
  rcases hp with hp | hp
  · exact Or.inl hp
  · exact Or.inr hp

theorem actual_classical_lower (x : BitVec 44) (h : ClassicalKernel x) :
    x = 0 ∨ 12 ≤ x.cpop.toNat := by
  rw [kernel_reconstruct x h]
  exact encoded_lower _
```

The next bridge packs actual coefficient functions into the bit-vector certificate and identifies population count with physical Hamming weight. The final theorem converts each source check equation into a certified row dot product before applying the lower bound.

Checked source: `FamilyBaseGraph.packWord_weight`; `FamilyBaseGraph.code_lower_twelve`.

```
theorem packWord_weight (x : Var → ZMod 2) : (packWord x).cpop.toNat = (CoverWeight.support
    x).card := by
  unfold packWord
  rw [BinaryFinite.pack44_weight]
  exact Finset.card_equiv varEquiv.symm (fun v => by simp [CoverWeight.support])

theorem code_lower_twelve (x : Var → ZMod 2) (hx : ∀ c, boundary x c = 0) (hn : x ≠ 0) :
    12 ≤ (CoverWeight.support x).card := by
  have hk : FamilyBase11.ClassicalKernel (packWord x) := fun c => (kernel_row x c).mpr (hx _)
  have hp : packWord x ≠ 0 := by
    intro hz
    apply hn
    funext v
    have hb := packWord_bit x (varEquiv v)
    rw [hz] at hb
    have hh : decide (x v ≠ 0) = false := by simpa using hb.symm
```

```
    exact not_ne_iff.mp (of_decide_eq_false hh)
  rcases FamilyClassical11.actual_classical_lower (packWord x) hk with hz | hl
  · exact False.elim (hp hz)
  · rwa [packWord_weight] at hl
```

C.5 The structural stopping bound

The classical Tanner-tree argument is credited to Kelley et al. [3] in the main text. Here is its complete checked degree-three, no-four/no-six implementation. It constructs six distinct selected variables explicitly; the no-cycle hypotheses rule out each potential collision.

Checked source: TannerSix.stopping_lower_six.

```
theorem stopping_lower_six (E : V → C → Prop) [DecidableRel E]
    (h4 : NoFour E) (h6 : NoSix E) (hd : ThreeChecks E)
    (T : Finset V) (hT : SmallStopping.Stopping E T) : 6 ≤ T.card := by
  obtain ⟨v,hv⟩ := hT.1
  obtain ⟨c,hci,hce⟩ := hd v
  have hp : ∀ i : Fin 3, ∃ w, w∈T ∧ w≠v ∧ E w (c i) :=
    fun i => partner E T hT v hv (c i) (hce i)
  choose w hwT hwv hwe using hp
  have hwi : Function.Injective w := by
    intro i j hij
    by_contra hne
    exact h4 v (w i) (c i) (c j) (hwv i).symm
      (fun h => hne (hci h)) (hce i) (hwe i) (by simpa [hij] using hwe j) (hce j)
  obtain ⟨a,b,hab,hac,hbc,hea,heb⟩ := two_more E hd (w 0) (c 0)
  have outside (d : C) (hd0 : d≠c 0) (hed : E (w 0) d) : ∀ i, d≠c i := by
    intro i hi
    by_cases hz : i=0
    · exact hd0 (hi.trans (congrArg c hz))
    · exact h4 v (w 0) (c 0) (c i) (hwv 0).symm
        (fun h => hz ((hci h).symm)) (hce 0) (hwe 0) (by simpa [hi] using hed) (hce i)
  have haout := outside a hac hea
  have hbout := outside b hbc heb
  obtain ⟨x,hxT,hxw,hxa⟩ := partner E T hT (w 0) (hwT 0) a hea
  obtain ⟨y,hyT,hyw,hyb⟩ := partner E T hT (w 0) (hwT 0) b heb
  have xv : x≠v := by
    intro h; subst x
    exact h4 v (w 0) (c 0) a (hwv 0).symm hac.symm (hce 0) (hwe 0) hea hxa
  have yv : y≠v := by
    intro h; subst y
    exact h4 v (w 0) (c 0) b (hwv 0).symm hbc.symm (hce 0) (hwe 0) heb hyb
  have xwi (i : Fin 3) (hi : i≠0) : x≠w i := by
    intro h
    exact h6 v (w 0) (w i) (c 0) a (c i)
      (hwv 0).symm (fun e => hi ((hwi e).symm)) (hwv i)
      hac.symm (haout i) (fun e => hi ((hci e)))
      (hce 0) (hwe 0) hea (by simpa [h] using hxa) (hwe i) (hce i)
  have ywi (i : Fin 3) (hi : i≠0) : y≠w i := by
    intro h
    exact h6 v (w 0) (w i) (c 0) b (c i)
      (hwv 0).symm (fun e => hi ((hwi e).symm)) (hwv i)
      hbc.symm (hbout i) (fun e => hi ((hci e)))
      (hce 0) (hwe 0) heb (by simpa [h] using hyb) (hwe i) (hce i)
```

```
  have hxy : x≠y := by
    intro h
    exact h4 (w 0) x a b hxw.symm hab hea hxa (by simpa [h] using hyb) heb
  have w01 : w 0≠w 1 := fun h => (by decide : (0:Fin 3)≠1) (hwi h)
  have w02 : w 0≠w 2 := fun h => (by decide : (0:Fin 3)≠2) (hwi h)
  have w12 : w 1≠w 2 := fun h => (by decide : (1:Fin 3)≠2) (hwi h)
  have hx1 := xwi 1 (by decide)
  have hx2 := xwi 2 (by decide)
  have hy1 := ywi 1 (by decide)
  have hy2 := ywi 2 (by decide)
  let S : Finset V := {v,w 0,w 1,w 2,x,y}
  have hS : S⊆T := by
    intro z hz
    simp only [S,Finset.mem_insert,Finset.mem_singleton] at hz
    rcases hz with rfl|rfl|rfl|rfl|rfl|rfl
    · exact hv
    · exact hwT 0
    · exact hwT 1
    · exact hwT 2
    · exact hxT
    · exact hyT
  have hc : S.card=6 := by
    simp [S,hwv 0,hwv 1,hwv 2,(hwv 0).symm,(hwv 1).symm,(hwv 2).symm,
      xv,xv.symm,yv,yv.symm,hxw,hxw.symm,hyw,hyw.symm,hx1,hx1.symm,hx2,hx2.symm,
      hy1,hy1.symm,hy2,hy2.symm,hxy,hxy.symm,w01,w01.symm,w02,w02.symm,w12,w12.symm]
  rw [← hc]
  exact Finset.card_le_card hS
```

C.6 Application to every positive multiple of eleven

Reduction modulo eleven supplies the additive map. The two premises of the generic theorem are discharged by the actual base kernel certificate and the independently proved structural stopping bound. This is the full family application, including the exponent-projection equality.

Checked source: FamilyCoverAlternative.classical_lower_twelve.

```
theorem classical_lower_twelve (hL : 11 ∣ L) (x : CyclicCover.Word (ZMod L))
    (hx : CyclicCover.Kernel DistanceFamily.liftE x) (hn : x≠0) :
    12≤(CoverWeight.support x).card := by
  let p : ZMod L →+ ZMod 11 := (ZMod.castHom hL (ZMod 11)).toAddMonoidHom
  have he : (fun i j => p (DistanceFamily.liftE i j)) = FamilyBaseGraph.e := by
    funext i j
    simp [p,DistanceFamily.liftE,FamilyBaseGraph.e]
  apply CoverDistance.lower_bound p DistanceFamily.liftE 12 6 _ _ x hx hn
  · intro y hy hny
    rw [he] at hy
    exact FamilyBaseGraph.code_lower_twelve y hy hny
  · intro T hT
    rw [he] at hT
    exact FamilyTreeStopping.base_stopping_six T hT
```

D Upper bounds, transpose distance and actual graph witnesses

D.1 A nonzero classical word of weight at most twenty

The coefficient definition below is (10). Kernel membership is proved for every positive L; nonzero membership uses reduction modulo eleven. The final step applies the physical support bound to the actual classical minimum. The helper `weight_le` and its support list are in the same linked source file.

Checked source: ClassicalUpper20.word; ClassicalUpper20.kernel; ClassicalUpper20.nonzero; ClassicalUpper20.classical_distance_upper.

```
def word (v : Fin 4 × ZMod L) : ZMod 2 :=
  ![delta 4 v.2+delta 5 v.2+delta 7 v.2+delta 8 v.2+delta 9 v.2+delta 11 v.2,
    delta 2 v.2+delta 3 v.2+delta 6 v.2+delta 7 v.2+delta 8 v.2+delta 11 v.2,
    delta 0 v.2+delta 2 v.2+delta 3 v.2+delta 6 v.2,
    delta 0 v.2+delta 4 v.2+delta 5 v.2+delta 9 v.2] v.1

theorem kernel : CyclicCover.Kernel DistanceFamily.liftE (word (L := L)) := by
  rintro ⟨i,t⟩
  fin_cases i <;>
    norm_num [CyclicCover.boundary,DistanceFamily.liftE,FamilyWords.exponent,word,
      Fin.sum_univ_succ,delta_sub,add_comm,add_left_comm,add_assoc]

theorem nonzero (hL : 11 ∣ L) : word (L := L)≠0 := by
  let p : ZMod L →+* ZMod 11 := ZMod.castHom hL (ZMod 11)
  have h2 : (2 : ZMod L)≠0 := by
    intro h
    have hh : (2 : ZMod 11)=0 := by simpa only [map_ofNat,map_zero] using congrArg p h
    exact (by decide : (2 : ZMod 11)≠0) hh
  have h3 : (3 : ZMod L)≠0 := by
    intro h
    have hh : (3 : ZMod 11)=0 := by simpa only [map_ofNat,map_zero] using congrArg p h
    exact (by decide : (3 : ZMod 11)≠0) hh
  have h6 : (6 : ZMod L)≠0 := by
    intro h
    have hh : (6 : ZMod 11)=0 := by simpa only [map_ofNat,map_zero] using congrArg p h
    exact (by decide : (6 : ZMod 11)≠0) hh
  have he : word ((2 : Fin 4),(0 : ZMod L))=1 := by
    simp [word,delta,h2,h3,h6,h2.symm,h3.symm,h6.symm]
  intro h
  have hh:=congrFun h ((2 : Fin 4),(0 : ZMod L))
  rw [he] at hh
  exact one_ne_zero hh

theorem classical_distance_upper (hL : 11 ∣ L) : DistanceFamily.classicalDistance L≤20 := by
  have h : DistanceFamily.ClassicalHasWeight L (CoverWeight.support (word (L := L))).card :=
    ⟨word,kernel,nonzero hL,rfl⟩
  exact (Nat.find_min' _ h).trans weight_le
```

D.2 The transpose period identity

The transpose boundary has the plus exponent $t + E_{ij}$. The following proof writes down the precise combination of shifted check equations in (11). The identity forces the third component to have period one without semisimplicity, oddness or a rank-preservation hypothesis.

Checked source: FamilyTranspose.boundary; FamilyTranspose.Kernel; FamilyTranspose.third_period.

```
def boundary (x : Fin 3 → ZMod L → ZMod 2) (j : Fin 4) (t : ZMod L) : ZMod 2 :=
  ∑ i : Fin 3, x i (t + (FamilyWords.exponent i j : ZMod L))

def Kernel (x : Fin 3 → ZMod L → ZMod 2) := ∀ j t, boundary x j t = 0

theorem third_period (x : Fin 3 → ZMod L → ZMod 2) (hx : Kernel x) (t : ZMod L) :
    x 2 (t+1) + x 2 (t+2) = 0 := by
  have h0_1 := hx 0 (t+1)
  have h0_4 := hx 0 (t+4)
  have h0_6 := hx 0 (t+6)
  have h0_7 := hx 0 (t+7)
  have h0_8 := hx 0 (t+8)
  have h0_9 := hx 0 (t+9)
  have h1_0 := hx 1 (t+0)
  have h1_2 := hx 1 (t+2)
  have h1_4 := hx 1 (t+4)
  have h1_8 := hx 1 (t+8)
  have h1_9 := hx 1 (t+9)
  have h2_3 := hx 2 (t+3)
  have h2_4 := hx 2 (t+4)
  have h3_0 := hx 3 (t+0)
  have h3_1 := hx 3 (t+1)
  have h3_2 := hx 3 (t+2)
  have h3_3 := hx 3 (t+3)
  have h3_4 := hx 3 (t+4)
  have h3_6 := hx 3 (t+6)
  have h3_7 := hx 3 (t+7)
  norm_num [boundary,FamilyWords.exponent,Fin.sum_univ_succ,add_assoc] at h0_1 h0_4 h0_6 h0_7
     h0_8 h0_9 h1_0 h1_2 h1_4 h1_8 h1_9 h2_3 h2_4 h3_0 h3_1 h3_2 h3_3 h3_4 h3_6 h3_7
  linear_combination (norm := ring_nf) h0_1 + h0_4 + h0_6 + h0_7 + h0_8 + h0_9 + h1_0 + h1_2
    + h1_4 + h1_8 + h1_9 + h2_3 + h2_4 + h3_0 + h3_1 + h3_2 + h3_3 + h3_4 + h3_6 + h3_7
  have htwo : (2 : ZMod 2) = 0 := by decide
  have hfour : (4 : ZMod 2) = 0 := by decide
  simp only [htwo,hfour,mul_zero,sub_zero,zero_sub,neg_zero]
```

The intervening lemmas in `TransposeConstant.lean` propagate period one to constancy of all three fibers. The full equivalence below includes the converse and the even-sum condition.

Checked source: FamilyTranspose.kernel_iff.

```
theorem kernel_iff (x : Fin 3 → ZMod L → ZMod 2) : Kernel x ↔
    (∀ i s, x i s = x i 0) ∧ x 0 0 + x 1 0 + x 2 0 = 0 := by
  constructor
  · intro hx
    refine ⟨all_constant x hx,?_⟩
    have h:=hx 0 0
    simpa [boundary,FamilyWords.exponent,Fin.sum_univ_succ,add_assoc] using h
  · rintro ⟨hc,hs⟩ j t
    unfold boundary
    simp_rw [hc]
    simpa [Fin.sum_univ_succ,add_assoc] using hs
```

D.3 Exact weight of every nonzero transpose word

The finite binary-triple fact `three_even_count` says that a nonzero even triple has exactly two ones. The proof below then counts the Cartesian product of those two occupied component types with all L fibers. The final conjunction includes existence, so the distance statement is not vacuous.

Checked source: FamilyTranspose.nonzero_weight; FamilyTranspose.distance_characterization.

```
theorem nonzero_weight (x : Fin 3 → ZMod L → ZMod 2) (hx : Kernel x) (hn : x ≠ 0) :
    (CoverWeight.support (word x)).card = 2*L := by
  have heven := (kernel_iff x).mp hx |>.2
  have hne : x 0 0 ≠ 0 ∨ x 1 0 ≠ 0 ∨ x 2 0 ≠ 0 := by
    by_contra h
    push Not at h
    apply hn
    funext i s
    rw [all_constant x hx i s]
    fin_cases i
    · exact h.1
    · exact h.2.1
    · exact h.2.2
  have hr : (Finset.univ.filter (fun i : Fin 3 => x i 0 ≠ 0)).card = 2 := by
    rw [Finset.card_eq_sum_ones,Finset.sum_filter]
    simpa [Fin.sum_univ_succ,Nat.add_assoc] using three_even_count (x 0 0) (x 1 0) (x 2 0)
    heven hne
  have hs : CoverWeight.support (word x) =
      (Finset.univ.filter (fun i : Fin 3 => x i 0 ≠ 0)) ×ˢ (Finset.univ : Finset (ZMod L)) :=
     by
    ext v
    simp [CoverWeight.support,word,all_constant x hx]
  rw [hs,Finset.card_product,hr]
  simp

theorem distance_characterization :
    (∀ x : Fin 3 → ZMod L → ZMod 2, Kernel x → x ≠ 0 →
      (CoverWeight.support (word x)).card = 2*L) ∧
    ∃ x : Fin 3 → ZMod L → ZMod 2, Kernel x ∧ x ≠ 0 :=
  ⟨nonzero_weight, pair,pair_kernel,pair_nonzero⟩
```

D.4 Girth exactly eight

The finite voltage tests and their reduction proofs exclude four- and six-cycles. To complete exact girth, one also needs a simple eight-cycle in the actual incidence. The following declarations give distinct vertices and all eight edges separately. The main text explains their assembly into the displayed cycle.

Checked source: FamilyGirth.eight_distinct; FamilyGirth.eight_edges; FamilyGirth.exact_eight_certificate.

```
theorem eight_distinct (hL : 11 ∣ L) :
    (checks (L := L)).Nodup ∧ (variableVertices (L := L)).Nodup := by
  have hl : 11 ≤ L := Nat.le_of_dvd (NeZero.pos L) hL
  letI : Fact (1<L) := ⟨by omega⟩
  simp [checks,variableVertices]

theorem eight_edges :
```

```
    VoltageCycles.Edge (FamilyNoShort.e L) (0,0) (0,0) ∧
    VoltageCycles.Edge (FamilyNoShort.e L) (1,0) (0,0) ∧
    VoltageCycles.Edge (FamilyNoShort.e L) (1,0) (1,-1) ∧
    VoltageCycles.Edge (FamilyNoShort.e L) (2,1) (1,-1) ∧
    VoltageCycles.Edge (FamilyNoShort.e L) (2,1) (0,1) ∧
    VoltageCycles.Edge (FamilyNoShort.e L) (1,1) (0,1) ∧
    VoltageCycles.Edge (FamilyNoShort.e L) (1,1) (1,0) ∧
    VoltageCycles.Edge (FamilyNoShort.e L) (0,0) (1,0) := by
  norm_num [VoltageCycles.Edge,FamilyNoShort.e,FamilyWords.exponent]

theorem exact_eight_certificate (hL : 11 ∣ L) :
    VoltageCycles.NoFourVoltage (FamilyNoShort.e L) ∧
    VoltageCycles.NoSixVoltage (FamilyNoShort.e L) ∧
    (checks (L := L)).Nodup ∧ (variableVertices (L := L)).Nodup :=
  ⟨FamilyNoShort.no_four hL,FamilyNoShort.no_six hL,(eight_distinct hL).1,(eight_distinct hL)
    .2⟩
```

D.5 Connectedness through actual Tanner walks

The normalized zero row connects column types within a fiber; the unit exponent supplies the following shift walk. The final proof combines variable-to-variable walks and the zero-column incidences to cover all variable/check endpoints. Both statements hold for every positive L.

Checked source: FamilyConnected.shift; FamilyConnected.connected.

```
theorem shift (s : ZMod L) : Reach (.inl (0,s)) (.inl (0,s+1)) := by
  apply (same_fiber 0 1 s).trans
  apply via (1,s) (0,s+1) (1,s+1) <;>
    norm_num [E,CyclicCover.Adjacent,e,FamilyWords.exponent]

theorem connected (a b : Vertex L) : Reach a b := by
  cases a with
  | inl v =>
    cases b with
    | inl w => exact vars_connected v w
    | inr c => exact (vars_connected v (0,c.2)).trans (.single (.vc (column_zero c)))
  | inr c =>
    cases b with
    | inl v => exact (Relation.ReflTransGen.single (.cv (column_zero c))).trans (
    vars_connected (0,c.2) v)
    | inr d => exact ((Relation.ReflTransGen.single (.cv (column_zero c))).trans
        (vars_connected (0,c.2) (0,d.2))).trans (.single (.vc (column_zero d)))
```

E Parameter transfer and the final source-distance statement

E.1 The arithmetic counterfamily

This complete implication displays every premise of the general parameter extension. An additive projection sends the parameters to $(1, 5)$; the conclusion includes the voltage exclusions, full partition exclusions, actual classical-kernel lower bound and source logical upper witness. The cyclic-congruence specialization is in the same source file.

Checked source: ParametricProjection.projected_counterfamily.

```
theorem projected_counterfamily (p : ZMod L →+ ZMod 11) (a b : ZMod L) (ha : p a=1) (hb : p
    b=5) :
    VoltageCycles.NoFourVoltage (ParametricWords.e a b) ∧
    VoltageCycles.NoSixVoltage (ParametricWords.e a b) ∧
    ¬ RowPairing (ParametricWords.e a b) ∧ ¬ ColumnPairing (ParametricWords.e a b) ∧
    (∀ x : CyclicCover.Word (ZMod L),
      (∀ c, ∑ v, GeneralLP.C (ParametricWords.e a b) c v*x v=0) →
      x≠0 → 12≤(CoverWeight.support x).card) ∧
    GeneralLP.LogicalX (ParametricWords.e a b) (ParametricWords.U a b) ParametricWords.V ∧
    (CoverWeight.support (GeneralLP.join (ParametricWords.U a b) ParametricWords.V)).card≤11
    :=
  ⟨no_four p a b ha hb,no_six p a b ha hb,no_row_pairing p a b ha hb,
    no_column_pairing p a b ha hb,actual_classical_lower p a b ha hb,
    ParametricWords.parametric_obstruction a b⟩
```

E.2 From block coordinates to every physical binary word

The disjoint-union coordinate map is surjective, so every physical word has two blocks. The first two equivalences transport logical-sector membership and exact support weight. The next two proofs identify the minima in both directions, using the corresponding weight equivalences. This is the bridge that prevents an accidentally restricted minimum.

Checked source: SourceWordBridge.block_weight_iff; SourceWordBridge.quantum_weight_iff; SourceWordBridge.quantum_distance_eq; SourceWordBridge.classical_distance_eq.

```
theorem block_weight_iff (n : Nat) : QuantumHasWeight L n ↔ SourceDistance.QuantumHasWeight
    L n := by
  constructor
  · rintro ⟨w,hw,hn⟩
    obtain ⟨u,v,rfl⟩ := SourceModel.join_surjective w
    exact ⟨u,v,hw,hn⟩
  · rintro ⟨u,v,hw,hn⟩
    exact ⟨SourceModel.join u v,hw,hn⟩

theorem quantum_weight_iff (n : Nat) : QuantumHasWeight L n ↔ DistanceFamily.
    QuantumHasWeight L n :=
  (block_weight_iff n).trans (SourceDistance.quantum_weight_iff n)

theorem quantum_distance_eq : quantumDistance L = DistanceFamily.quantumDistance L := by
  apply Nat.le_antisymm
  · exact Nat.find_min' _ ((quantum_weight_iff _).mpr (Nat.find_spec (DistanceFamily.
    quantum_exists (L := L))))
  · exact Nat.find_min' _ ((quantum_weight_iff _).mp (Nat.find_spec (quantum_exists (L := L)))
    )
```

```
theorem classical_distance_eq : classicalDistance L = DistanceFamily.classicalDistance L :=
    by
  apply Nat.le_antisymm
  · exact Nat.find_min' _ ((SourceDistance.classical_weight_iff _).mpr (Nat.find_spec (
    DistanceFamily.classical_exists (L := L))))
  · exact Nat.find_min' _ ((SourceDistance.classical_weight_iff _).mp (Nat.find_spec (
    classical_exists (L := L))))
```

E.3 The final distance drop

All substantive ingredients now occur as checked dependencies. The last theorem rewrites the actual source minima, applies the quantum witness and classical lower theorem, and concludes strict separation. The other clauses of Theorem 1.1 are supplied by the graph, transpose and upper-word proofs above; they are not extra assumptions of this distance statement.

Checked source: SourceWordBridge.source_distance_drop.

```
theorem source_distance_drop (hL : 11 ∣ L) :
    quantumDistance L ≤ 11 ∧ 12 ≤ classicalDistance L ∧ quantumDistance L <
    classicalDistance L := by
  rw [quantum_distance_eq,classical_distance_eq]
  exact ⟨DistanceFamily.quantum_distance_upper,DistanceFamily.classical_distance_lower hL,
    DistanceFamily.distance_drop hL⟩
```

Reproducing the checks. The fifty original modules and their full import closure were rebuilt against the pinned cached dependencies as described in Section 6. The manuscript declaration audit checks every named declaration linked here, including those added to the exposition. The listing generator verifies that each source equals its published Git blob, and that each printed declaration is a complete source range. The archive contains that generator, the range/hash manifest, the declaration audit and the portable rebuild instructions. The exact typesetting extracts add no axioms or proof modules.

F Complete current and immutable Lean URLs

The following index includes every admitted proof, certificate and research import. All immutable links use commit `98eb91cd9049e6a88887a5ba79150ad96a4a3e53`. The declaration links in the text and preceding appendices go directly to source line ranges. Current links may evolve; the immutable sources match the audited hashes. The source archive and companion proof guide give the complete SHA-256 ledger and logs.

Source file: `BinaryFinite.lean`
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/BinaryFinite.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/BinaryFinite.lean

Source file: `BinaryWords.lean`
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/BinaryWords.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/BinaryWords.lean

Source file: `CSS11.lean`
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/CSS11.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/CSS11.lean

Source file: `ClassicalDistance11.lean`
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/ClassicalDistance11.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/ClassicalDistance11.lean

Source file: `ClassicalEnum11.lean`
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/ClassicalEnum11.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/ClassicalEnum11.lean

Source file: `ClassicalUpper20.lean`
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/ClassicalUpper20.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/ClassicalUpper20.lean

Source file: `Counterexample11.lean`
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/Counterexample11.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/Counterexample11.lean

Source file: `CoverDistance.lean`
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/CoverDistance.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/CoverDistance.lean

Source file: `CoverWeight.lean`
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/CoverWeight.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/CoverWeight.lean

Source file: `CyclicCover.lean`
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/CyclicCover.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/CyclicCover.lean

Source file: FamilyBaseDistance.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FamilyBaseDistance.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FamilyBaseDistance.lean

Source file: FamilyBaseGraph.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FamilyBaseGraph.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FamilyBaseGraph.lean

Source file: FamilyBounds.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FamilyBounds.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FamilyBounds.lean

Source file: FamilyConnected.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FamilyConnected.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FamilyConnected.lean

Source file: FamilyCoverAlternative.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FamilyCoverAlternative.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FamilyCoverAlternative.lean

Source file: FamilyDistance11.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FamilyDistance11.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FamilyDistance11.lean

Source file: FamilyDotRows11.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FamilyDotRows11.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FamilyDotRows11.lean

Source file: FamilyEnum11.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FamilyEnum11.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FamilyEnum11.lean

Source file: FamilyGirth.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FamilyGirth.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FamilyGirth.lean

Source file: FamilyNoShort.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FamilyNoShort.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FamilyNoShort.lean

Source file: FamilyRing.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FamilyRing.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FamilyRing.lean

Source file: FamilySource11.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FamilySource11.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FamilySource11.lean

Source file: FamilyStopping11.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FamilyStopping11.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FamilyStopping11.lean

Source file: FamilyTheorem.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FamilyTheorem.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FamilyTheorem.lean

Source file: FamilyTranspose.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FamilyTranspose.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FamilyTranspose.lean

Source file: FamilyTreeStopping.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FamilyTreeStopping.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FamilyTreeStopping.lean

Source file: FamilyWords.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FamilyWords.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FamilyWords.lean

Source file: FinalAudit.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/FinalAudit.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/FinalAudit.lean

Source file: GeneralLP.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/GeneralLP.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/GeneralLP.lean

Source file: GeneralLPPhysical.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/GeneralLPPhysical.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/GeneralLPPhysical.lean

Source file: Hypotheses11.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/Hypotheses11.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/Hypotheses11.lean

Source file: ParametricProjection.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/ParametricProjection.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/ParametricProjection.lean

Source file: ParametricRing.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/ParametricRing.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/ParametricRing.lean

Source file: ParametricSource.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/ParametricSource.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/ParametricSource.lean

Source file: ParametricSpecialization.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/ParametricSpecialization.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/ParametricSpecialization.lean

Source file: ParametricWords.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/ParametricWords.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/ParametricWords.lean

Source file: QuantumRows11.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/QuantumRows11.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/QuantumRows11.lean

Source file: QuantumWitness11.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/QuantumWitness11.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/QuantumWitness11.lean

Source file: SmallStopping.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/SmallStopping.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/SmallStopping.lean

Source file: Source11.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/Source11.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/Source11.lean

Source file: SourceCoordinates.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/SourceCoordinates.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/SourceCoordinates.lean

Source file: SourceDistance.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/SourceDistance.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/SourceDistance.lean

Source file: SourceModel.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/SourceModel.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/SourceModel.lean

Source file: SourceSectors.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/SourceSectors.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/SourceSectors.lean

Source file: SourceSymmetry.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/SourceSymmetry.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/SourceSymmetry.lean

Source file: SourceWordBridge.lean
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/SourceWordBridge.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/SourceWordBridge.lean

Source file: `TannerSix.lean`
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/TannerSix.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/TannerSix.lean

Source file: `TransposeConstant.lean`
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/TransposeConstant.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/TransposeConstant.lean

Source file: `TransposeDistance.lean`
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/TransposeDistance.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/TransposeDistance.lean

Source file: `VoltageCycles.lean`
Current: https://github.com/alectionary/vibe-math/blob/master/formalization/distance-preservation/VoltageCycles.lean
Immutable: https://github.com/alectionary/vibe-math/blob/98eb91cd9049e6a88887a5ba79150ad96a4a3e53/formalization/distance-preservation/VoltageCycles.lean

G Reference dates and evidence boundaries

The chronology in Section 1 uses the manuscript date 25 October 2026 in UTC. Calendar subtraction from 10 March 2025 gives 594 days and from 10 June 2026 gives 137 days. Only year precision is used for the inspected ISIT 2004 source; its age is therefore stated approximately. The arXiv histories inspected for [1, 2] list v1 only. These checks do not establish global unresolved status. Both July preprints were inspected in the v1 copies recorded in the evidence ledger, which carry an internal manuscript date of 24 August 2026. The corresponding deltas are 87 and 62 days. The current abstract page for [4] additionally lists v2 dated 2 October 2026; that later version was identified during report preparation and is retained as a version-comparison gap, not treated as a screened full text. The comparison above is explicitly to the inspected v1 theorem.

The complete investigation ledger is `prior-art.json` and the semantic translation record is `SOURCE-TO-PROOF.txt`. The prior-art audit includes primary papers, author references, public GitHub code and signed-in Edge searches of GitHub and X. It records queries, known-positive controls, caps and inspected results. Later native-browser access failed when the Mac locked; indexed search did not replace complete native coverage. Some discovered leads were available only as snippets and were not used as theorem evidence. The precise unresolved leads and access limitations remain in that ledger. The searches support concrete source comparisons, not a claim that no earlier result exists anywhere.

The supporting research record separates general Lean proofs, concrete Lean certificates, exact finite computations and sampled diagnostics. In particular, negative C++ searches are external computations. A completion-shortcut bug was repaired, with an old-fails/new-passes regression, 1,235 small full-word comparisons and replay of every original saved input. Only corrected replay records support current negative search statements. These computations are absent from the theorem dependency closure. The separate SAT attempt returned `UNKNOWN`, not a distance certificate. None of these investigation details changes the proved counterfamily.

The companion `README.txt` gives artifact layout and reproduction commands. The manuscript source archive contains the fifty admitted Lean sources, their checked build records, the dependency pins, the source-to-proof guide, evidence ledgers and the manuscript. Content hashes identify the local review snapshot. The investigation duration and subsequent manuscript preparation are recorded there separately; they are not source chronology claims.

References

[1] N. Raveendran, D. Declercq and B. Vasić, *On the Minimum Distances of Finite-Length Lifted Product Quantum LDPC Codes*, arXiv:2503.07567v1, 10 March 2025. Full text; Eq. (9), Definitions 2–5, Theorem 2, Sections V-B and VI.

[2] R. Freij-Hollanti, K. D. Morris and P. Šapokaitė, *Graphical Analysis of Lifted Product Code Constructions*, arXiv:2606.11987v1, 10 June 2026. Full text; sector graph isomorphism, connectivity and Section 5.

[3] C. Kelley, D. Sridhara, J. Xu and J. Rosenthal, *Pseudocodeword Weights and Stopping Sets*, Proceedings of the IEEE International Symposium on Information Theory, 2004. Theorem III.1 and its following note; Sections II–III. Author-hosted paper.

[4] A. Bhardwaj et al., *High-rate qLDPC processors*, arXiv:2607.28795v1, first submitted 30 July 2026; inspected copy internally dated 24 August 2026. Appendix G, Theorem 8 and Remark 12. Inspected version; version history (later v2 identified).

[5] Y. Hirasaki and J. Y. Lee, *Lifting Lifted Product Codes*, arXiv:2607.28621v1, first submitted 30 July 2026; inspected copy internally dated 24 August 2026. Section II and Corollary 6. Full text.